

Kari Calvey

Module 3 SQL Query Analysis

Question 1:

Question: How many entries in the database are from applicants who applied for Fall 2026?

Result: 11,110

SQL Query:

```
SELECT COUNT(*)  
  
FROM applicants  
  
WHERE LOWER(term) = 'fall 2026';
```

Explanation:

This query counts all applicant records where the term is Fall 2026. Using LOWER() makes the comparison less sensitive to differences in capitalization.

Question 2:

Question: Among entries that provide a nationality classification, what percentage are international students?

Result: 48.74%

SQL Query:

```
SELECT  
  
    100.0 *  
  
    COUNT(*) FILTER (  
  
        WHERE LOWER(TRIM(us_or_international)) = 'international'  
  
    )  
  
    /  
  
    NULLIF(COUNT(*), 0)  
  
FROM applicants  
  
WHERE us_or_international IS NOT NULL  
  
    AND TRIM(us_or_international) <> '';
```

Explanation:

This query counts international applicants and divides that number by all applicants who provided a usable nationality classification. Blank and missing nationality values are excluded from the calculation.

Question 3:

Question: What are the average GPA, GRE Quantitative, GRE Verbal, and GRE Analytical Writing scores of applicants who provide each metric?

Results:

Average GPA: 3.76

Average GRE Quantitative: 190.06

Average GRE Verbal: 159.73

Average GRE Analytical Writing: 5.75

SQL Query:

```
SELECT
    AVG(gpa),
    AVG(gre),
    AVG(gre_v),
    AVG(gre_aw)
FROM applicants;
```

Explanation:

This query calculates the average of each academic metric separately. PostgreSQL's AVG() function ignores NULL values, so applicants only contribute to an average when they provided that specific value.

Question 4:

Question: What is the average GPA of American applicants who applied for Fall 2026?

Result: 3.75

SQL Query:

```
SELECT AVG(gpa)
FROM applicants
WHERE LOWER(term) = 'fall 2026'
    AND LOWER(TRIM(us_or_international)) = 'american'
    AND gpa IS NOT NULL;
```

Explanation:

This query filters the data to Fall 2026 applicants classified as American and then calculates the average GPA for records that contain a GPA.

Question 5:

Question: What percentage of Fall 2025 entries are acceptances?

Result: 55.41%

SQL Query:

```
SELECT
    100.0 *
    SUM(
        CASE
            WHEN LOWER(status) LIKE '%accept%'
            THEN 1
            ELSE 0
        END
    )
    /
    COUNT(*)
FROM applicants
WHERE LOWER(term) = 'fall 2025';
```

Explanation:

This query counts Fall 2025 records whose status contains an indication of acceptance and divides that number by the total number of Fall 2025 entries.

Question 6:

Question: What is the average GPA of accepted applicants who applied for Fall 2026?

Result: 3.75

SQL Query:

```
SELECT AVG(gpa)
FROM applicants
WHERE LOWER(term) = 'fall 2026'
    AND LOWER(status) LIKE '%accept%'
    AND gpa IS NOT NULL;
```

Explanation:

This query filters the records to accepted Fall 2026 applicants who provided a GPA and calculates the average GPA for that group.

Question 7:

Question: How many entries are from applicants who applied to Johns Hopkins University for a master's degree in Computer Science?

Result: 21

SQL Query:

```
SELECT COUNT(*)
FROM applicants
WHERE LOWER(program) LIKE '%computer science%'
      AND (
          LOWER(program) LIKE '%johns hopkins university%'
          OR LOWER(program) LIKE '%jhu%'
      )
      AND LOWER(degree) LIKE '%master%';
```

Explanation:

This query searches the original program field for Computer Science and either Johns Hopkins University or JHU. It also limits the results to master's degree applicants.

Question 8:

Question: Using the original downloaded fields, how many Fall 2026 accepted PhD Computer Science applicants applied to Georgetown University, MIT, Stanford University, or Carnegie Mellon University?

Result: 1

SQL Query:

```
SELECT COUNT(*)
FROM applicants
WHERE LOWER(term) = 'fall 2026'
      AND LOWER(status) LIKE '%accept%'
      AND LOWER(degree) = 'phd'
      AND LOWER(program) LIKE '%computer science%'
```

```
AND (  
    LOWER(program) LIKE '%georgetown university%'  
    OR LOWER(program) LIKE '%massachusetts institute of technology%'  
    OR LOWER(program) LIKE '%mit%'  
    OR LOWER(program) LIKE '%stanford university%'  
    OR LOWER(program) LIKE '%carnegie mellon university%'  
);
```

Explanation:

This query uses the original program field and filters records by Fall 2026, acceptance status, PhD degree, Computer Science, and one of the four required universities.

Question 9:

Question: Repeat Question 8 using the LLM-generated university and program fields. Compare the results.

Results:

Original-field count: 1

LLM-field count: 1

Difference: +0

SQL Query:

```
SELECT COUNT(*)  
FROM applicants  
WHERE LOWER(term) = 'fall 2026'  
    AND LOWER(status) LIKE '%accept%'  
    AND LOWER(degree) = 'phd'  
    AND LOWER(llm_generated_program) LIKE '%computer science%'  
    AND (  
        LOWER(llm_generated_university) LIKE '%georgetown university%'  
        OR LOWER(llm_generated_university) LIKE '%massachusetts institute of technology%'  
        OR LOWER(llm_generated_university) LIKE '%mit%'  
        OR LOWER(llm_generated_university) LIKE '%stanford university%'
```

```
OR LOWER(llm_generated_university) LIKE '%carnegie mellon university%'
```

```
);
```

Explanation:

This query repeats Question 8 but uses the LLM-generated university and program fields instead of the original program field. In my dataset, both methods returned one record, so the difference between the two counts was zero.

Question 10:

Question: Which five universities have the highest number of applications in the dataset?

Result:

University of California: 812

Columbia University: 794

New York University: 588

University of Pennsylvania: 515

University of Michigan: 493

SQL Query:

```
SELECT
```

```
    llm_generated_university,
```

```
    COUNT(*) AS application_count
```

```
FROM applicants
```

```
WHERE llm_generated_university IS NOT NULL
```

```
    AND TRIM(llm_generated_university) <> ''
```

```
GROUP BY llm_generated_university
```

```
ORDER BY application_count DESC
```

```
LIMIT 5;
```

Explanation:

This query groups applicants by their LLM-generated university name and counts how many records belong to each university. The results are sorted from highest to lowest and limited to the top five.

Question 11:

Question: What is the average GPA of applicants to Johns Hopkins University?

Result: 3.82

SQL Query:

```
SELECT AVG(gpa)
FROM applicants
WHERE LOWER(llm_generated_university) LIKE '%johns hopkins%';
```

Explanation:

This query finds applicants whose LLM-generated university field contains Johns Hopkins and calculates the average GPA for those records.